WEBRUM-01: Web RUM Fundamentals

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 1 of 10 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

Real User Monitoring (RUM) captures the actual experience of users interacting with your web applications. Unlike synthetic monitoring, which simulates user interactions from controlled locations, RUM measures real browser performance, user behavior, and errors as they happen. Dynatrace RUM uses a lightweight JavaScript agent (OneAgent RUM injection) to automatically capture page loads, user actions, XHR/fetch calls, JavaScript errors, and Core Web Vitals — all flowing into Grail for DQL analysis.

Sprint 1.337 (April 2026): `frontend.name` Now Permission-Relevant

Sprint 1.337 added `frontend.name` as a **permission-relevant field** on RUM metrics, sessions, and Smartscape entities in Grail. This means IAM policies can now scope record-level and field-level access by frontend application, eliminating the need to split RUM data into separate buckets purely for access control.

Practical impact for WEBRUM authors:

- When you instrument a new application, choose `frontend.name` deliberately — it now drives **both observability** (queries, dashboards, segments) **and access control** (which teams can see the data).
- Stable, hyphenated identifiers work best: `checkout-web`, `account-web`, `public-marketing`. Avoid spaces, version numbers in the name, or environment suffixes (those belong on `dt.security_context` or environment fields).
- Existing tenants automatically gain the field — no re-instrumentation required. Update IAM policies (see **IAM-04 § Sprint 1.337**) to take advantage.

Table of Contents

1. How Dynatrace RUM Works
2. RUM vs Synthetic Monitoring
3. RUM Data Model
4. Exploring User Sessions
5. Exploring User Actions
6. Session and Action Counts
7. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	At least one web application with RUM injection active
Permissions	`storage:events:read`, `storage:entities:read`
Data	At least 24 hours of RUM session data

1. How Dynatrace RUM Works

Dynatrace captures real user experience through a **JavaScript agent** that is automatically injected into web pages by OneAgent. The injection process works as follows:

1. **OneAgent instruments the web server** — When a browser requests a page, OneAgent on the server-side injects a small JavaScript snippet into the HTML response.
2. **The JavaScript agent loads** — The injected snippet bootstraps the full RUM JavaScript agent from the Dynatrace CDN or your ActiveGate.
3. **Browser events are captured** — The agent hooks into browser APIs (Performance API, Navigation Timing, Resource Timing, PerformanceObserver) to capture timing data, user interactions, and errors.
4. **Data is sent to Dynatrace** — Captured data is batched and sent as beacons to the Dynatrace cluster via the beacon endpoint.

5. **Data lands in Grail** — Sessions, actions, and errors are stored as structured records queryable via DQL.

Injection Methods

Method	How It Works	Best For
Automatic injection	OneAgent modifies HTML responses at the web server	Traditional server-rendered apps
Manual injection	Developer adds the RUM script tag manually	CDN-hosted content, SPAs with no server-side agent
Agentless RUM	JavaScript tag added without full OneAgent	Environments where OneAgent cannot be deployed

> **Note:** Where the web server runs OneAgent, automatic injection is the default path — OneAgent in full-stack monitoring mode *"by default automatically injects the RUM JavaScript"* into the pages it serves, with no code change. In community practice it is the preferred method for that reason; manual injection and agentless RUM are for serving tiers where OneAgent cannot run.
>
> **Sources:** Configure automatic injection in RUM Classic (DT docs).

2. RUM vs Synthetic Monitoring

RUM and synthetic monitoring are complementary approaches — each answers different questions:

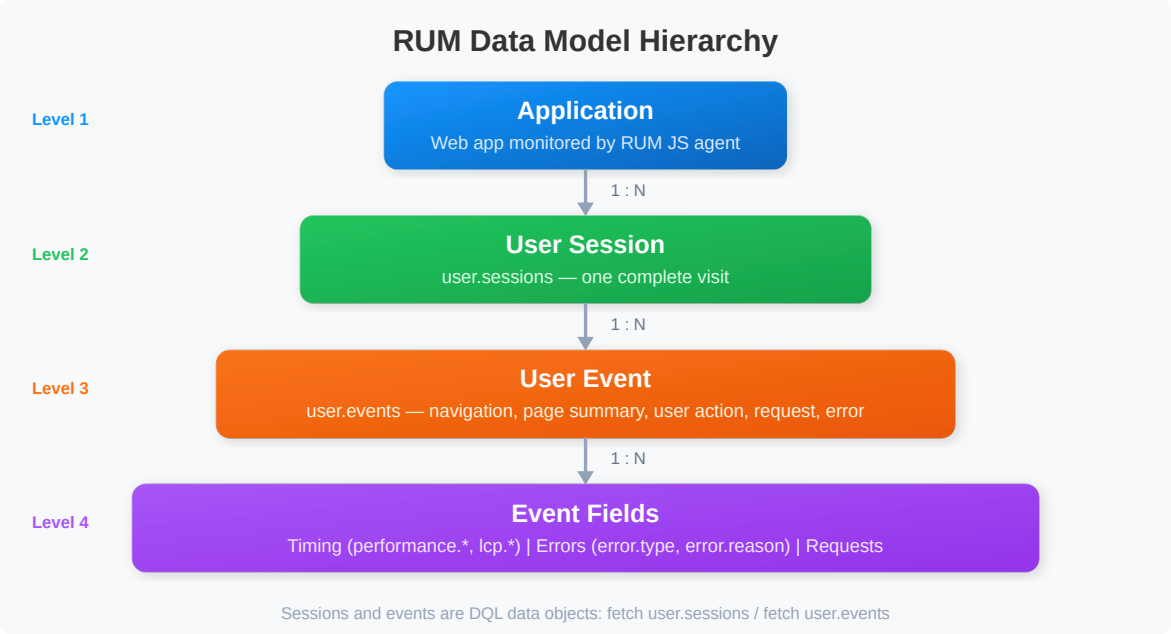
Aspect	RUM	Synthetic
Data source	Real user browsers	Simulated browsers from controlled locations
Coverage	Only when real users visit	Runs on schedule, even at 3 AM
Variability	High — reflects real network conditions, devices, geographies	Low — consistent execution environment
Baseline	Organic — based on actual usage patterns	Fixed — ideal for SLA monitoring

Aspect	RUM	Synthetic
Error detection	Catches errors users actually encounter	Catches errors in scripted flows
Performance data	Real-world load times with real content	Clean-room load times (no cache, extensions)
Geographic insight	Wherever real users are located	Only from configured locations

> **Tip:** Use synthetic for availability SLAs and baseline benchmarks. Use RUM for understanding the actual user experience and identifying issues affecting real users.

3. RUM Data Model

Dynatrace RUM data is organized into a hierarchical model:



Key Data Objects

DQL Data Object	Description	Key Fields
`user.sessions`	A complete user visit from first page to last	`dt.rum.session.id`, `dt.rum.user_type`, `duration`, `user_action_count`, `error.count`
`user.events`	A single event — navigation, page summary, user action, request, interaction — told apart by `characteristics.has_*` flags	`page.detected_name`, `user_action.type`, `performance.load_event_end` (navigations), `lcp.start_time` (page summaries)
`user.events` (`characteristics.has_error == true`)	JavaScript exception, failed request or CSP violation	`error.reason`, `error.type` (`exception` / `request` / `csp`), `error.source`
`smartscapeNodes "FRONTEND"`	The monitored application **entity** — one node per configured web or mobile app	`name`, `frontend.name`, `frontend.type`, `id_classic`

The Application Entity: `FRONTEND`

Everything else in the table above is *session and action data*. The last row is the **entity** — the configured application those sessions belong to. On the classic surface that was `dt.entity.application`; on Smartscape it is the `FRONTEND` node type (model `dt.smartscape.frontend`, titled *Digital Experience Frontend*), which holds web **and** mobile applications, discriminated by `frontend.type`.

```
smartscapeNodes "FRONTEND"
| filter frontend.type == "web"
| fields name, frontend.name, id, id_classic, tags
| sort name asc
```

SaaS 1.344 (released 07/27/2026, with a **staged tenant rollout from 07/29/2026**) is what makes this the primary entity surface: the Digital Experience apps — Experience Vitals, Users & Sessions, Error Inspector, Synthetic — moved so that their queries and filters "primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` and

`dt.rum.application.*`". Verify 1.344 has reached your tenant before depending on the app-side change. The node type is queryable via DQL either way, and classic `fetch dt.entity.application` still works — keep it as a genuine fallback during the rollout.

Three details worth knowing before you use the node:

- **`name` vs `frontend.name`.** `name` is the display name (`Beta - Elevate`); `frontend.name` is the normalized form determined at ingest (`Beta_-_Elevate`). Use `name` for reporting, `frontend.name` when matching against ingest-side configuration.
- **`id_classic` is the join key.** It holds the `APPLICATION-*` id (or `MOBILE_APPLICATION-*` for a mobile app), so dashboards and saved queries still written against `dt.entity.application` can be reconciled with migrated ones.
- **`frontend.type` filters correctly but is absent from the model's own `fields` array.** The Semantic Dictionary's field list is an index, not a completeness guarantee — do not conclude the field is missing because it is not enumerated.

> **This is orthogonal to the vocabulary question in WEBRUM-09 — read them as answering different questions.** WEBRUM-09 asks which **session-field** vocabulary your tenant serves: classic RUM on Grail (`userActionCount`, `totalErrorCount`, `browserFamily`) or New RUM (`user_action_count`, `error.count`, `browser.name`). That answer governs every session and action query in the rest of this series. `FRONTEND` is a change to the **entity** vocabulary and is independent of it — it is how you enumerate and identify applications whichever session model your tenant serves. Answer WEBRUM-09's question for your session queries; use `FRONTEND` for your entity queries. Neither decision constrains the other.

Frontend-to-Backend Topology (SaaS 1.344)

Two 1.344 changes connect this RUM data to the backend side of the platform:

- **The Smartscape service dependency graph now displays `FRONTEND` nodes**, letting you "drill down the call path from user-facing applications to the services they depend on, all in one view" — frontend and service topology were previously separate pictures.
- **Distributed tracing can explore frontend events and sessions without switching apps.**

The Semantic Dictionary exposes the linkage itself as `frontend.link` — *"a frontend-backend tracing link that connects a span to a frontend user event and/or user session"*, explicitly distinct from a span link, which references other spans. Two cautions before building on it: the field is `frontend.link` (dotted), and its declared stability is `experimental` — not yet a foundation for a production dashboard or alert.

Until 1.344 reaches your tenant, the working correlation path is unchanged — and it keeps working afterwards. For **same-origin** web requests, linking a user action to its distributed trace works out of the box: the browser sends the RUM cookies, which OneAgent on a supported first tier evaluates. **Cross-origin** XHR actions are not linked automatically; the fix is either cookies on cross-origin calls or the optional `x-dtc` header, which the RUM JavaScript adds only once you enable it, because the receiving endpoints must first be configured to accept it (WEBRUM-02 covers the SPA instrumentation side). Session-level joins on the application and session identifiers supplement both.

> **The topology side belongs to the tracing series.** SPANS-04 is the better home for the service-dependency-graph and trace-exploration mechanics; this note exists so a reader arriving from RUM knows the linkage exists and what it is called.
>
> **Sources:** Dynatrace SaaS release notes 1.344 (DT docs), Link cross-origin XHR user actions and their distributed traces (DT docs) — *"Its addition needs to be explicitly enabled because the endpoints that handle the cross-origin requests need to be configured to accept the header."* **Dictionary:** `frontend.link` (`experimental` , type `record`), read 09/28/2026.

Session Types

Every session and user event carries `dt.rum.user_type`. Its documented values are lowercase:

`dt.rum.user_type`	Description
`real_user`	*"The user event was produced by a real user."*
`robot`	*"The user event was produced by a bot user."*
`synthetic`	*"The user event was produced by a synthetic test."*

> **Note:** Always filter for `dt.rum.user_type == "real_user"` when analyzing genuine user experience to exclude bots and synthetic executions. The classic `userType == "REAL_USER"` form reads null on New RUM data and filters out every session without an error.
>
> **Sources:** User events — semantic dictionary (DT docs) — *"The user event was produced by a real user."*
>
> **Dictionary:** `dt.rum.user_type` (`experimental` , example `real_user`), read 10/05/2026.

Tagging this producer — primary Grail tags on RUM data

RUM data *"enters Dynatrace through the rumagent ingest source, which offers no at-source tag configuration."* Ingest enrichment rules don't reach it either, so team, app and security-context values are set with an OpenPipeline **DQL processor** keyed on `frontend.name`:

- **Add it in every RUM scope.** User events, Smartscape events, Events, Davis events, Business events and Timeseries — *"Each scope is independent."* Put the processor in a base pipeline (pipeline groups) so the tag exists before any team pipeline runs.
- **Guard it.** Match on `isNotNull(frontend.name) and isNull(primary_tags.team)`, so a value set earlier is not overwritten.
- **User sessions are different.** `frontend.name` is an array there, *"because a web session can span multiple web frontends"*. Match with `arrayContains(frontend.name, "<frontend>")`; a session that crossed two teams' frontends carries both team values, so query it with `matchesValue(primary_tags.team, "<team>")`, not `==`.

The same path applies to mobile apps (OneAgent for Mobile). FAQ-26 § 5.7 has the full pattern.

> **Sources:** Enrich RUM telemetry with primary Grail fields and tags (DT docs) — *"RUM data from web frontends (RUM JavaScript) and mobile apps (OneAgent for Mobile) enters Dynatrace through the rumagent ingest source, which offers no at-source tag configuration."* and *"Each scope is independent."*; Apply primary Grail tags across an example AWS estate (DT docs) — RUM step, User sessions matching condition.

4. Exploring User Sessions

Let's start by examining what a user session looks like in Grail. A session represents a single visit — from the first page load to when the user closes the tab or the session times out (30 minutes of inactivity by default).

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM). Classic
camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so
expand it before grouping)
//                  primary_tags.application exists only if your OpenPipeline
adds it (corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries; web_vitals.time_to_first_byte
where populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request
wait/redirect phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the documented values are "real_user" / "robot" / "synthetic"
(lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Explore recent user sessions — sample 10 sessions to see available fields
fetch user.sessions, from:-1h
| filter dt.rum.user_type == "real_user"
| fieldsKeep dt.rum.session.id, frontend.name, dt.rum.user_type, duration,
user_action_count, error.count, geo.city.name, geo.country.name, os.name,
browser.name
| sort duration desc
| limit 10
```

The query above returns the 10 longest sessions from the last hour. Each record includes the session ID, application name, duration, action count (number of interactions), error count, and geographic/device information.

Session Duration Distribution

Understanding how long sessions last helps identify engagement patterns:

```
// Session duration distribution — bucket sessions by duration range
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| fieldsAdd duration_sec = duration / 1s
| fieldsAdd duration_bucket = if(duration_sec &lt; 10, "&lt; 10s",
    else: if(duration_sec &lt; 30, "10-30s",
    else: if(duration_sec &lt; 60, "30-60s",
    else: if(duration_sec &lt; 300, "1-5 min",
    else: "&gt; 5 min"))))
| summarize session_count = count(), by:{duration_bucket}
| sort session_count desc
```

5. Exploring User Actions

User actions represent individual interactions within a session. In New RUM each one is a `user.events` record with `characteristics.has_user_action == true`, and `user_action.type` says what triggered it. The web values are:

`user_action.type`	**Classic RUM equivalent**	**Documented meaning**
`hard_navigation`	Load	*"The user action was triggered by a hard navigation (full page load)."*
`soft_navigation`	Route change	*"The user action was triggered by a soft navigation that led to a view change."*
`same_view`	XHR	*"The user action was triggered by a user interaction, followed by an XHR or fetch request."*
`api`	Custom	*"The user action was reported via the Dynatrace API."*

> **Sources:** User actions — semantic dictionary (DT docs) — *"The user action was triggered by a soft navigation that led to a view change."*

Let's explore the most common user actions:

> `dt.rum.application.id` **is being deprecated (Semantic Dictionary 1.348, released 08/25/2026).** It moves from `stable` to `deprecated`. This series used to group by it; the per-application queries now group and filter by `frontend.name`, the stable primary field

> *"present on every user event and user session"* (corrected 10/14/2026). The change is still worth knowing if your own queries use the ID.
>
> **Deprecated is not removed.** Every query in this series keeps working; a deprecated field is a signal about where to point *new* work, not a break. Verified 09/09/2026 on the validation tenant, which is still pre-1.348 and returns `stable` for this field — so nothing has changed there yet.
>
> The changelog names no successor. Rather than guess one, check the dictionary when 1.348 reaches your tenant:
> `fetch dt.semantic_dictionary.fields | filter name == "dt.rum.application.id" | fields name, stability, description`
>
> **Sources:** Enrich RUM telemetry with primary Grail fields and tags (DT docs) — *"This primary field is present on every user event and user session; its value comes from the frontend nodes defined in Dynatrace."* **Dictionary:** `frontend.name` (`stable`), `dt.rum.application.id` (`stable`), read 10/14/2026.

```
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**, but
was written
// against names that are null on New RUM data, so these cells returned nothing
while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.has_page_summary
== true
//                                       (was characteristics.classifier — "not
intended for query usage", SD 1.349)
//   action.type                       -&gt; user_action.type
(hard_navigation | soft_navigation | same_view | api)
//   action.name                       -&gt; page.detected_name
//   LCP / CLS                         -&gt;
web_vitals.largest_contentful_paint / web_vitals.cumulative_layout_shift
//                                       (stable — the fields Dynatrace's own
DQL examples use), falling back to
//                                       the experimental lcp.start_time /
cls.value (corrected 10/14/2026)
//   app.name                          -&gt; frontend.name (stable; an array on
user.sessions)
// UNITS DIFFER BETWEEN THE TWO FAMILIES. web_vitals.largest_contentful_paint is
a DURATION, so
// `/ 1ms` turns it into milliseconds; lcp.start_time is a PLAIN NUMBER already
in milliseconds,
// and dividing it by 1ms yields null. The CWV cells therefore read
// coalesce(web_vitals.largest_contentful_paint / 1ms, lcp.start_time) and
// coalesce(web_vitals.cumulative_layout_shift, cls.value), against the
2500/4000 ms and 0.1/0.25 thresholds.
// Why both: on the validation tenant every user event comes from the synthetic
engine, which fills
// lcp.* / cls.* but not web_vitals.* (0 of 243,494 page summaries in 30 days,
10/14/2026). That
// is a property of synthetic-monitor data, not a sign that web_vitals.* is
rarely populated.
// Top 20 user actions by count in the last hour (user-action events only — most
user.events
// records are requests, which carry no user_action.type)
fetch user.events, from:-1h
| filter characteristics.has_user_action == true
| summarize {action_count = count(), avg_duration = avg(duration)}, by:
{page.detected_name, user_action.type}
| sort action_count desc
| limit 20
```

Page Load Actions with Timing Breakdown

For load-type actions, Dynatrace captures detailed timing milestones:

Select `user.events` with the `characteristics.has_*` flags, not `characteristics.classifier`. The User events semantic-dictionary page describes `characteristics.classifier` as the single characteristic picked by priority when an event has several, and says it is *"Used for internal optimization when storing the data and not intended for query usage."* **Semantic Dictionary 1.349** (published 09/04/2026) removes it from the user-event models, with a **staged tenant rollout** — a tenant that has not reached 1.349 still returns the field, so a classifier filter keeps working until the change arrives and then silently matches nothing. The stable flags (`has_page_summary`, `has_navigation`, `has_error`, `has_request`, …) are the documented selectors and work on both sides of the change. **Semantic Dictionary 1.350** (published 09/21/2026) re-lists `characteristics.classifier` under the Basic user-event model, still `experimental`, and the User events page still calls it *"not intended for query usage"* — so the flags above remain the selectors.

The flags can **overlap** where the classifier picked one winner. A navigation that also carried an error was classified `error`, so `classifier == "navigation"` silently dropped it; `characteristics.has_navigation == true` keeps it. On the validation tenant (24 h, 09/28/2026) that is **7,387** navigations against **6,235** — the 1,152 recovered page loads have full navigation timing. For page summaries the two filters select the same 7,268 events.

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type: "Load" -&gt; "hard_navigation", "RouteChange" -&gt;
"soft_navigation",
//                     "Xhr" -&gt; "same_view", "Custom" -&gt; "api" (documented
values, corrected 10/05/2026)
//   connection.type         -&gt; no web RUM equivalent
(network.connection.type is OneAgent for Mobile
//                              only; network.protocol.name is the OSI protocol,
e.g. "http")
// THE EVENT CHARACTERISTIC MATTERS AS MUCH AS THE FIELD: navigation-timing
fields
// (performance.dom_interactive, performance.load_event_end) live on navigation
events
// (characteristics.has_navigation) and are unpopulated on page summaries
// (characteristics.has_page_summary), so a page-summary filter silently empties
them.
// ttfb.* is the opposite — it lives on page summaries. Select with the stable
has_* flags, not
// characteristics.classifier (corrected 09/28/2026): the docs call classifier
"not intended for
// query usage" and Semantic Dictionary 1.349 removes it from the user-event
models (re-listed in SD 1.350, still not for query use).
// Page load timing breakdown — average timings for the top 10 pages
fetch user.events, from:-1h
| filter characteristics.has_navigation == true
| summarize {action_count = count(),
    avg_duration = avg(duration),
    avg_dom_interactive = avg(performance.dom_interactive),
    avg_load_event = avg(performance.load_event_end)},
    by:{page.detected_name}
| sort action_count desc
| limit 10
```

6. Session and Action Counts

Tracking session and action volume over time reveals traffic patterns, peak hours, and trend changes.

```
// Session count over the last 24 hours, bucketed by hour
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| makeTimeseries session_count = count(), interval:1h
```

```
// Action volume by type over the last 24 hours — user actions only; without the
filter,
// requests and other events dominate as a null user_action.type series
fetch user.events, from:-24h
| filter characteristics.has_user_action == true
| makeTimeseries action_count = count(), interval:1h, by:{user_action.type}
```

```
// Sessions and actions per application — high-level overview
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {session_count = count(),
    total_actions = sum(user_action_count),
    total_errors = sum(error.count),
    avg_duration = avg(duration)},
    by:{frontend.name}
| sort session_count desc
```

7. Summary and Next Steps

In this notebook, we covered:

- **How RUM works** — JavaScript agent injection, browser event capture, beacon delivery to Grail
- **RUM vs synthetic** — Complementary approaches for full UX visibility
- **RUM data model** — Sessions, actions, and errors as hierarchical records in Grail
- **Basic DQL queries** — Exploring sessions, actions, timing breakdowns, and volume trends

Next Steps

- **WEBRUM-02: SPA Instrumentation** — Deep dive into Single-Page Application monitoring with Angular, React, and Vue.js
- **WEBRUM-03: Core Web Vitals** — Track LCP, INP, and CLS with DQL

References

- Dynatrace RUM Overview
- RUM JavaScript version (DT docs)
- Monitor web performance with DQL (DT docs)
- User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*
- Semantic Dictionary changelog 1.349 (DT docs)
- Semantic Dictionary changelog 1.350 (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.